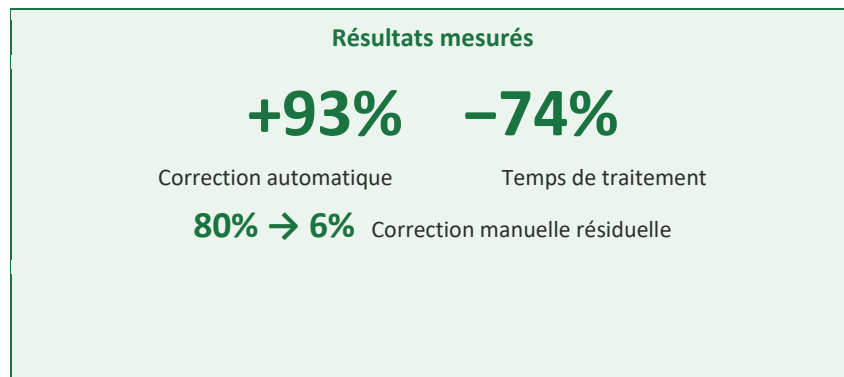


Pipeline Multi-Agents IA

*Validation et correction automatique de fichiers CSV
pour le Process Mining (PMapp)*



Auteur	BOUKHRIS Mohamed Lyazid
Tutrice académique	Dr. Gema Ibañez Sanchez
Institution	ITACA Institute, Universitat Politècnica de València
Groupe de recherche	SABIEN Research Group
Période	19 mai 2025 – 8 août 2025 (3 mois)

1. Institution d'accueil et contexte de recherche

1.1 L'Institut ITACA

L'Institut ITACA (Instituto de Aplicaciones de las Tecnologías de la Información y de las Comunicaciones Avanzadas) est un centre de recherche spécialisé dans les technologies de l'information et de la communication, rattaché à l'Universitat Politècnica de València en Espagne. Sa mission est de mettre l'innovation scientifique au service de la société, avec un fort accent sur la santé, le bien-être, les systèmes de communication et les transports. L'institut rassemble plus de cent chercheurs et participe à de nombreux projets nationaux et européens.

1.2 Le groupe de recherche SABIEN

Mon stage s'est déroulé au sein du groupe SABIEN, spécialisé dans l'application des technologies de l'information à la santé. L'équipe développe des outils digitaux pour accompagner les professionnels de santé et est particulièrement active dans :

- L'analyse de données et le process mining pour les processus hospitaliers
- Les systèmes d'aide à la décision pour cliniciens et managers
- Les projets collaboratifs avec des hôpitaux et des partenaires européens

Ma mission portait directement sur la qualité des données utilisées pour le process mining, en collaboration avec des hôpitaux partenaires en Espagne, au Portugal et en Suède.

2. Vue d'ensemble du projet

2.1 Contexte et problème

Les techniques de process mining reposent sur la qualité des données d'événements exportées par les systèmes d'information hospitaliers, généralement sous forme de fichiers CSV ou eCSV. En pratique, ces fichiers bruts souffrent de nombreux problèmes :

Problèmes identifiés sur les fichiers hospitaliers

- Encodages hétérogènes selon les pays (UTF-8, Latin-1, Windows-1252)
- Délimiteurs inconsistants (virgule, point-virgule, tabulation)
- Noms de colonnes différents entre hôpitaux (patient_id, PatientID, id_patient)
- Valeurs catégorielles inconsistantes (Discharge, discharge, DISCHARGE)
- Séquences temporelles incohérentes — patient « sortant » avant d'« entrer »
- Doublons, valeurs aberrantes, champs obligatoires manquants

Impact opérationnel : Les data managers passaient 80% de leur temps à corriger manuellement ces fichiers. Seulement 20% du temps était consacré à l'analyse à forte valeur ajoutée.

2.2 Objectif de la mission

Concevoir et déployer un pipeline de validation et correction automatique capable de valider la structure des fichiers, mapper les colonnes, normaliser les valeurs, valider la cohérence temporelle, et produire une décision actionnable : READY / WARNING / REJECTED — le tout alimentant un dashboard Power BI pour le suivi des KPIs.

2.3 Benchmark des solutions existantes

Démarche : Avant toute conception, j'ai réalisé un benchmark comparatif pour justifier chaque choix technologique — notamment pour les agents nécessitant de la sémantique.

Solution	Capacités	Raison d'exclusion
OpenAI API	Embeddings puissants, LLM	Données médicales : transfert vers serveurs US non conforme RGPD. Coût variable non maîtrisable.
Mistral AI	Modèle français, souveraineté relative	Même contrainte de transfert. Déploiement local non disponible à l'époque.
Azure Cognitive	NLP Microsoft cloud	Appel API externe : latence + coût + dépendance réseau dans un pipeline batch.
spaCy	NLP open source performant	Stack C#/.NET : incompatible sans couche de traduction coûteuse.
ML.NET ✓	Embeddings C#, déploiement local	RETENU : Natif C#, embarquable Docker, aucun appel externe, données médicales jamais transférées, coût nul.

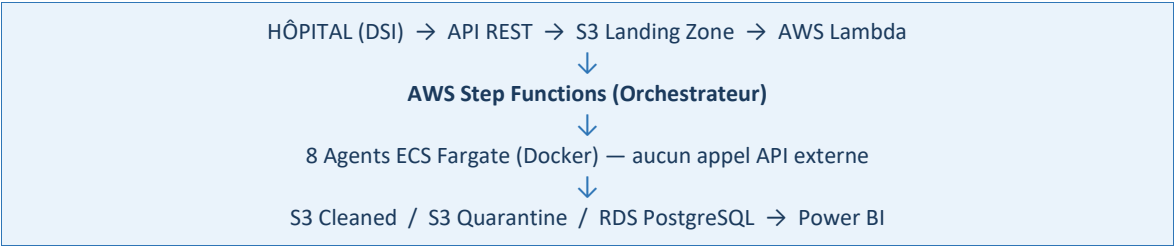
Conclusion : ML.NET pour les agents sémantiques. Algorithmes déterministes pour tout ce qui ne nécessite pas de modèle. Ce choix garantit la souveraineté des données, élimine les coûts variables et maximise la performance.

2.4 Stack technique

Couche	Technologies
Développement	C# / .NET — logique métier, parsing, pipeline de validation
IA sémantique	ML.NET (Microsoft) — embeddings pré-entraînés, chargés localement
Interface	WPF — application desktop interactive
Orchestration	AWS Step Functions — enchaînement des agents, gestion des erreurs
Conteneurisation	Docker + AWS ECS Fargate — facturation à l'usage réel
Infrastructure	AWS S3, Lambda, RDS PostgreSQL, CloudWatch + Terraform
Reporting	Power BI — dashboard KPIs avec mesures DAX
Gestion projet	Scrum (sprints 2 sem.), Jira, coordination Zoom avec 3 hôpitaux

3. Architecture du pipeline multi-agents

3.1 Vue d'ensemble



Chaque agent est un conteneur Docker indépendant sur AWS ECS Fargate. AWS Step Functions les enchaîne dans l'ordre, passe l'état via un objet JSON partagé et gère les erreurs et relances. Aucun modèle IA ne fait d'appel API externe — tous les modèles sont embarqués dans les conteneurs.

3.2 L'Orchestrateur — AWS Step Functions

ORCHESTRATEUR — AWS Step Functions	
Rôle	Piloter le pipeline : déclencher les agents dans l'ordre, gérer les erreurs, relances et état global
Déclenchement	Trigger automatique via AWS Lambda dès qu'un fichier est déposé en S3
Erreurs	Retry automatique configurable, puis branchement vers S3 Quarantine
Punchline	« L'orchestrateur transforme 8 agents indépendants en un pipeline cohérent, mesurable et contrôlé. »

3.3 Les 8 agents

Agent 1 — Contrôle structurel du fichier

Agent 1 — Contrôle structure & format

Problème	3 hôpitaux, 3 pays : encodages hétérogènes (UTF-8, Latin-1, Windows-1252) et délimiteurs inconsistants
Technologie	C# natif + bibliothèque CharDetSharp (détection d'encodage par analyse de bytes)
Modèle	Règle déterministe — résultat 100% prévisible, coût nul
Décision	Fichier vide ou corrompu → REJECTED immédiat

Agent 2 — Mapping des colonnes (sémantique)**Agent 2 — Mapping des colonnes & validation du schéma eCSV**

Problème	Chaque hôpital nomme ses colonnes différemment : patient_id, PatientID, id_patient
Technologie	ML.NET (Microsoft) — modèle d'embeddings pré-entraîné chargé localement
Modèle	Vectorisation des noms de colonnes + similarité cosinus avec le schéma eCSV. Seuil configurable : au-dessus → mapping auto, en dessous → WARNING
Intégration	Modèle embarqué dans Docker au build — aucun appel API, aucun transfert de données
Décision	Colonne obligatoire manquante → REJECTED

Agent 3 — Validation temporelle**Agent 3 — Timestamps & cohérence des séquences**

Problème	Un patient ne peut pas « sortir » avant d'« entrer ». Timestamps partiels ou séquences inversées = analyse PMAApp invalidée
Technologie	C# natif — bibliothèques date/heure .NET
Modèle	Règle déterministe : parsing multi-formats, tri chronologique par case_id, détection des inversions, minimum 2 événements par trace
Décision	Inversion temporelle ou trace < 2 événements → WARNING

Agent 4 — Nettoyage et déduplication**Agent 4 — Nettoyage, déduplication & normalisation**

Problème	Doublons, colonnes numériques avec du texte, valeurs aberrantes, labels avec espaces résiduels
Technologie	C# natif — hashing MD5/SHA, regex, contrôles de plage
Modèle	Algorithmes classiques : hashing ligne par ligne (plus rapide et moins coûteux qu'un modèle), regex, plages configurables

Décision	Doublons supprimés automatiquement et logués dans le rapport
-----------------	--

Agent 5 — Clustering sémantique (agent hybride)

Agent 5 — Clustering sémantique (hybride)	
Problème	Discharge, discharge, DISCHARGE : PMAApp les traite comme 3 activités distinctes et fausse l'analyse
Technologie	Hybride : Distance de Levenshtein (C# natif) + ML.NET embeddings
Modèle	Niveau 1 — Levenshtein < seuil : normalisation automatique des variantes simples (casse, fautes légères). Niveau 2 — cosinus ML.NET > seuil : clustering sémantique. La valeur la plus fréquente devient la référence normalisée
Pourquoi hybride	Règle classique pour les cas simples (rapide, sans coût), modèle pour les cas ambigus (précision maximale)

Agent 6 — Scoring et décision finale

Agent 6 — Score de confiance & décision READY/WARNING/REJECTED	
Problème	Il faut une décision actionnable et traçable, pas une liste brute d'erreurs techniques
Technologie	C# natif — scoring pondéré
Modèle	Pondération configurable par criticité d'erreur. Score > seuil haut → READY. Entre seuils → WARNING. Score < seuil bas → REJECTED

Agent 7 — Génération du rapport

Agent 7 — Rapport d'erreurs JSON	
Problème	Sans rapport lisible, le data manager ne peut pas corriger son fichier rejeté
Technologie	System.Text.Json natif .NET
Contenu	Résumé (statut, score), liste des erreurs par agent, corrections appliquées, recommandations
Stockage	S3 Reports + tables RDS PostgreSQL pour alimentation Power BI

Agent 8 — Persistance et Power BI

Agent 8 — Écriture RDS PostgreSQL & alimentation Power BI	
Problème	Sans persistance structurée, pas de mesure de la valeur créée dans le temps
Technologie	AWS SDK for .NET — RDS PostgreSQL
Tables	file_runs, rule_violations, corrections_applied, hospital_reference

Valeur	KPIs temps réel : taux de correction, gains de temps, performance par hôpital
--------	---

3.4 Tableau récapitulatif des agents

#	Agent	Technologie	Type
Orch.	Orchestrateur	AWS Step Functions	Service AWS natif
1	Structure & format	CharDetSharp + C#	Règle déterministe
2	Mapping colonnes	ML.NET — embeddings	Modèle IA local
3	Timestamps & traces	Parsing dates + tri C#	Règle déterministe
4	Nettoyage	Hashing MD5 + regex C#	Algorithme classique
5	Clustering sémantique	Levenshtein + ML.NET	Hybride
6	Scoring & décision	Pondération configurable	Règle déterministe
7	Rapport JSON	System.Text.Json .NET	Sérialisation
8	Persistence BDD	AWS SDK + PostgreSQL	Écriture BDD

4. Workflow de l'application desktop

L'application CSV Structural Validator offre une interface desktop WPF pour une validation interactive étape par étape. Cette interface constituait la première version du projet, avant le déploiement du pipeline cloud automatisé.

4.1 Étape 1 — Sélection et chargement du fichier

L'utilisateur sélectionne un fichier CSV ou eCSV depuis la fenêtre principale. L'interface confirme le chargement et active l'étape suivante.

4.2 Étape 2 — Validation du délimiteur

Aperçu des premières lignes avec le délimiteur détecté automatiquement. L'utilisateur peut le confirmer ou le modifier manuellement.

4.3 Étape 3 — Détection des types de colonnes

L'application propose un type inféré pour chaque colonne (string, date, integer). L'utilisateur ajuste si nécessaire — étape critique pour la bonne interprétation des timestamps par PMAApp.

4.4 Étape 4 — Valeurs fréquentes et inconsistantes

Calcul des distributions de fréquence par colonne. Identification des valeurs candidates à la normalisation (Arrival vs. Arrive). L'utilisateur garde le contrôle sur les remplacements appliqués.

4.5 Étape 5 — Synthèse et export

Bilan complet : types validés, valeurs détectées, erreurs résiduelles. Export du CSV nettoyé prêt pour PMAApp et du rapport TXT détaillant toutes les opérations effectuées.

5. Résultats et impact

5.1 Résultats mesurés

Indicateur	Avant	Après
Temps consacré à la correction manuelle	80%	6%
Taux de correction automatique	—	+93%
Réduction du temps de traitement global	Référence	-74%
Temps disponible pour l'analyse process	20%	94%

5.2 Dashboard Power BI

Le dashboard comporte 5 pages :

- **PM Readiness** : PM Readiness
- PM Readiness — taux READY/WARNING/REJECTED, évolution hebdomadaire
- Top erreurs Pareto — fréquence des erreurs par type
- Performance par hôpital — comparaison ES / PT / SE
- Gains mesurés — avant/après, courbe d'amélioration sur les sprints
- Amélioration continue — erreurs rétro → intégrées au backlog → sprints

5.3 Approche FinOps — Maîtrise des coûts AWS

- **ECS Fargate** : facturation uniquement au temps d'exécution réel
- **S3 Lifecycle Policies** : archivage vers S3 Glacier après 30 jours (~60% de réduction du stockage)
- **Lambda** : facturation à l'invocation, quasi gratuit pour le volume de fichiers
- **CloudWatch** : alertes budget configurées, monitoring en temps réel
-

6. Développement durable et RSE

6.1 Considérations environnementales

L'Institut ITACA est situé sur le campus de l'Universitat Politècnica de València, dans des bâtiments à faible consommation énergétique. L'institut optimise l'utilisation des ressources et mutualise le matériel entre groupes de recherche. Mon pipeline contribue à cet objectif : une réduction de 74% du temps de traitement se traduit directement par une réduction des ressources computationnelles consommées.

6.2 Dimension sociale et inclusive

Le stage s'est déroulé dans un contexte multilingue (anglais, espagnol) avec des partenaires dans 3 pays. ITACA favorise la diversité culturelle, l'intégration des personnes en situation de handicap et la collaboration internationale.

6.3 Contribution aux enjeux sociétaux

Le projet améliore la qualité des données utilisées dans les études de process mining en santé, contribuant à mieux comprendre et optimiser les parcours de soins des patients.

7. Compétences développées

7.1 Compétences techniques

- **Pipeline IA multi-agents** : conception, déploiement et orchestration de 8 agents AWS
- **Benchmark technologique** : évaluation comparative (OpenAI, Mistral, Azure, ML.NET) selon coût, souveraineté et performance
- **ML.NET et embeddings** : modèles pré-entraînés pour le mapping sémantique et le clustering

- **Architecture AWS** : Step Functions, ECS Fargate, Lambda, S3, RDS PostgreSQL, CloudWatch, Terraform
- **Power BI** : modélisation DAX, connexion RDS, dashboard multi-pages avec KPIs
- **Qualité des données** : impact sur les analyses de process mining en santé

7.2 Compétences en gestion de projet

- **Scrum international** : pilotage de sprints de 2 semaines, coordination avec 3 hôpitaux dans 3 pays
- **Jira** : backlog, user stories, story points, planning poker, vélocité
- **Communication multilingue** : réunions Zoom en anglais avec des DSI hospitaliers
- **Autonomie et livraison** : gestion complète du cycle, de l'analyse des besoins à la livraison

7.3 Compétences personnelles

- Adaptation à un environnement de recherche international (Valencia, Espagne)
- Communication professionnelle en anglais et en espagnol
- Rigueur analytique : arbitrages coût / performance / souveraineté
- Orientation résultats : mesure systématique de la valeur créée

8. Conclusion

Au cours de ce stage à l'Institut ITACA, j'ai conçu et déployé un pipeline multi-agents IA pour la validation et la correction automatique de fichiers CSV dans un contexte de process mining médical international.

Le projet a abouti à :

- Un pipeline de 8 agents orchestrés par AWS Step Functions, conteneurisés sans dépendance API externe
- Un benchmark technologique justifiant chaque choix architectural
- Des résultats mesurés : +93% de correction automatique, -74% de temps de traitement
- Un dashboard Power BI pour le suivi des KPIs et la mesure des gains
- Une documentation complète livrée à l'équipe SABIEN pour la mise en production

Au-delà des résultats techniques, ce projet m'a appris une chose fondamentale : l'IA n'a de valeur que si elle est choisie avec discernement. Utiliser un modèle là où un algorithme déterministe suffit, c'est un coût inutile. Utiliser une règle là où la sémantique est indispensable, c'est une limite bloquante.